

CA2 React Assessment – Student Evaluation Guidelines

1. Purpose

This document explains how your React application will be evaluated and the mandatory requirements you must follow during the assessment.

2. Application Deployment (Vercel)

- Your application must be deployed using **Vercel**
- Submit:
 - Vercel URL (mandatory)
 - GitHub repository (optional)

Important Rules

- Application must be publicly accessible
- All routes must work correctly on the deployed URL
- Localhost URLs will not be accepted

3. Routes (Provided in Question Paper)

The required routes will be explicitly mentioned in the question paper.

Example:

```
/orders  
/orders/:id  
/filter  
/stats
```

Important

- You must implement exactly the same routes
- Routes are case-sensitive
- Do not modify route paths

4. Test IDs (Provided in Question Paper)

The required data-testid values will be specified in the question paper.

Example:

```
<div data-testid="total-orders"></div>  
<div data-testid="delivered-orders"></div>  
<div data-testid="cancelled-orders"></div>  
<div data-testid="order-item"></div>
```

Important Rules

- You must use the exact test IDs as given
- Do not rename, modify, or remove them

Warning

If data-testid values do not match exactly, your application may fail evaluation even if the logic is correct.

5. Stats Component Requirement (Mandatory)

All evaluation-related outputs must be implemented inside the **/stats route component only**.

This includes:

- All required data-testid elements
- window.appState

Example:

```
<div data-testid="total-orders">30</div>
<div data-testid="delivered-orders">18</div>
<div data-testid="cancelled-orders">6</div>
window.appState = {
  totalOrders,
  deliveredOrders,
  cancelledOrders
};
```

Important Rules

- Must be implemented only in the /stats page
- Must update dynamically based on data
- Do not hardcode values
- Do not place these in other components

6. Global State Exposure (Mandatory)

You must expose computed values using:

```
window.appState = {
  totalOrders,
  deliveredOrders,
  cancelledOrders
};
```

Note

- This requirement will be mentioned in the question paper
- Values must be computed dynamically
- Hardcoded values will not be accepted

7. Evaluation Flow

Your application will be evaluated automatically using the following process:

```
Step 1: Load your deployed application
Step 2: Navigate to required routes
Step 3: Access the /stats page
Step 4: Read UI using data-testid
Step 5: Read window.appState
Step 6: Validate outputs
```

8. What the Evaluator Checks

Data Rendering

- Correct number of items (*-item)

Computed Values

- Total values
- Derived counts

Functionality

- Filtering
- Status updates
- Data consistency

9. Common Mistakes to Avoid

- Incorrect route paths
- Wrong data-testid values
- Missing window.appState
- Hardcoding values
- Not handling invalid data
- Using static/local data instead of API
- Placing stats logic outside /stats page

10. Development Rules

- Use Context as single source of truth
- Use Reducer for all updates
- Use map, filter, reduce for logic
- Do not store derived values in state



11. Final Checklist Before Submission

- Vercel link is working
- Routes match question paper exactly
- Test IDs match question paper exactly
- /stats page contains all required outputs
- window.appState is available and correct
- All values are computed dynamically
- No console errors

Final Note

Routes and test IDs will be provided in the question paper. All evaluation-related outputs must be implemented in the /stats component. You must follow these instructions exactly for successful evaluation.